

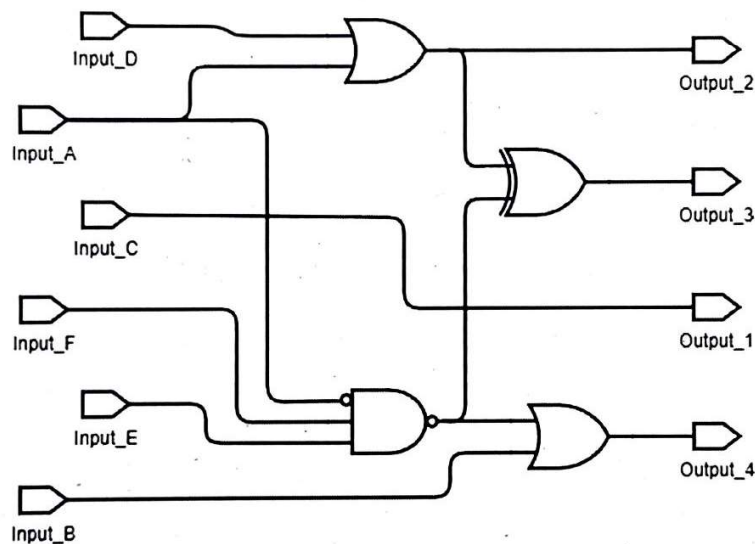
Aufgabe 3 (Mikrocontroller GPIO, 30 Punkte)

Spezifikation

Der Mikrocontroller soll eine SPS-Funktionalität realisieren (SPS: Speicherprogrammierbare Steuerung). Die SPS-Aufgabe ist dabei wie folgt definiert:

- 6 Digitale SPS-Eingänge (Input_A bis Input_F)
- 4 Digitale SPS-Ausgänge (Output_1 bis Output_4)
- Es ist keine bestimmte SPS-Zykluszeit erforderlich, der Mikrocontroller arbeitet in einer Endlosschleife mit der sich dann zwangsläufig ergebenden Zykluszeit
- Alle Eingangssignale sollen in engem zeitlichen Zusammenhang eingelesen werden.
- Alle Ausgangssignale sollen in engem zeitlichen Zusammenhang ausgegeben werden.
- Insbesondere solle es keine zeitliche Überschneidung zwischen Eingabe- und Ausgabephase geben.

Folgendes Digitalschaltung soll per C-Programm nachgebildet und wiederkehrend in einer Endlosschleife ausgeführt werden:



Nachfolgende Tabelle zeigt die Zuordnung der logischen Signale zu den GPIO-Ports des Mikrocontrollers TM4C1294NCPDTI. Beachten Sie, dass die grau eingefärbten Port-Bits weder in ihrer Datenrichtung noch in ihrem Dateninhalt verändert werden dürfen. Sonst käme es zu Seiteneffekten mit anderen Prozessen.

	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0
Belegung Port K (R9*)	Input_D		Input_A	Input_C	Input_E		Input_B	Input_F
Belegung Port L (R10*)	Output_3	Output_1	Output_2	Output_4				

*Referenz für die Clock Gating Control Einstellung

Aufgabe 3 (Fortsetzung)

Aufgabe 3.1) (Port Initialisierung)

Nachfolgend sind Ausschnitte des Codes abgebildet. Ergänzen Sie den Code entsprechend der Aufgabenstellung. Für die SPS-Anwendung nicht benutzte Bit-Felder dürfen nicht verändert werden.

```
#include "inc/tm4c1294ncpdt.h"
int wt = 0; // global variable for
void sps_ports_init(void)
{
    // for TIVA connected Launchpad
    SYSTCTL_RCGCGPIO_R
    wt++ ;           //

    // Port K: SPS inputs
    GPIO_PORTK_DEN_R
    GPIO_PORTK_DIR_R

    // Port L: SPS outputs
    GPIO_PORTL_DEN_R
    GPIO_PORTL_DIR_R
}
```

Ergänzen Sie:

- ← diesen Kommentar
- ← Zuweisungsoperator und Zuweisungswert
- ← Kommentar
- ← Zuweisungsoperatoren und Zuweisungswerte
- ← Zuweisungsoperatoren und Zuweisungswerte

Aufgabe 3.2)

Auf der nächsten Seite ist ein Programmgerüst gegeben, in das Sie bitte den für die SPS-Realisierung notwendigen Code eintragen.

Zur Vermeidung zeitlich überlappender Ein- und Ausgabephasen ist der Programmablauf streng strukturiert in: SPS-Eingabe – SPS-Verarbeitung – SPS-Ausgabe.

Innerhalb der SPS-Verarbeitung darf kein direktes Einlesen bzw. Ausgeben von SPS-Signalen erfolgen. Die Verteilung der Teilaufgaben ist also wie folgt:

- **SPS-Eingabe:** Einlesen der SPS-Eingänge in Variablen passenden Typs. Zur Sicherstellung eines engen zeitlichen Zusammenhangs sollte nur ein einziger Lesezugriff auf den GPIO-Datenport erfolgen. Somit verbietet sich die Anwendung des "GPIO_PORTK_DATA_BITS_R" Makros, sondern es soll die "klassische" Methode der Maskierung zur Bestimmung der Zustände der Eingangssignale angewendet werden (Eingangsquelle ist somit das Makro: "GPIO_PORTK_DATA_R"). Diese Methode begünstigt auch die Kompatibilität mit anderen Mikrocontrollern, die den Adressbus nicht zur Maskierung heranziehen können.
- **SPS-Verarbeitung:** Geschäftslogik zur Nachbildung der Gatterschaltung. Legen Sie dazu gegebenenfalls Variablen für interne Signale an.
- **SPS-Ausgabe:** Ausgabe der Ergebnisvariablen auf die SPS-Ausgänge. Zur Sicherstellung eines engen zeitlichen Zusammenhangs sollte nur ein einziger Schreibzugriff auf den GPIO-Datenport erfolgen. Somit verbietet sich die Anwendung des "GPIO_PORTL_DATA_BITS_R" Makros, sondern es soll die "klassische" Read-Modify-Write Methode mit Maskierung für die Ausgabe angewendet werden (Datenziel ist somit das Makro: "GPIO_PORTL_DATA_R"). Diese Methode begünstigt auch die Kompatibilität mit anderen Mikrocontrollern.

Aufgabe 3 (Fortsetzung)

Aufgabe 3.2) (Programmierung)

Die unter Aufgabe 3.1) ergänzten Programmteile sind auch hier gültig und müssen nicht noch einmal aufgeführt aber ggf. angewendet werden. Kommentieren Sie Ihren Code!

```
void main(void){
    // variable definition and initialization
```

```
// SPS Initialization
```

```
while (1) {
    // perform SPS input
```

```
// perform SPS functional processing
```

```
// perform SPS output
```

} }